

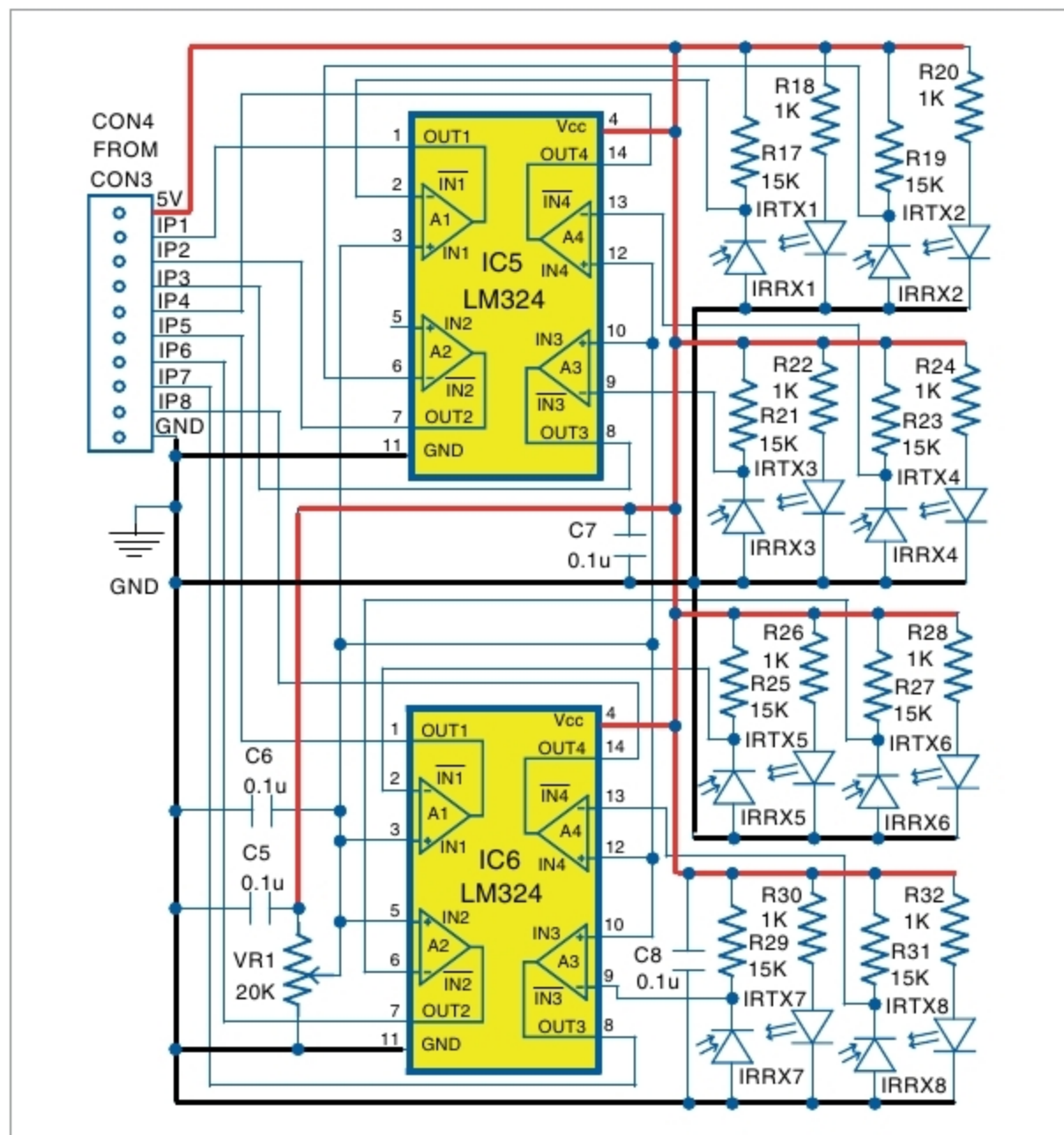


Fig. 3: Sensor circuit of the line follower

$P = K_p$ multiplied by error value
 $I = K_i$ multiplied with previous integral error value
 $D = K_d$ multiplied by derivative value

Note that the constants K_p , K_i , and K_d are not the same for every line follower. One must assume approximate values for the above three constants every time and then compile and burn the code into the microcontroller for testing the behaviour of the line follower. This is a very tedious job, and a lot of time is wasted during testing.

The application of the above formula is simplified using the function `void runPID()` in the source code. The constant values K_p , K_i , K_d , delays and loops required to control the line follower can be modified by just pressing three buttons.

The current set values can be viewed on the OLED display, and the same are automatically saved to EEPROM of the microcontroller, which makes testing the line follower with the PID algorithm simple and easier.

Circuit and working

The circuit diagram of the custom PID line follower has two parts: the main circuit and sensor circuit. The main circuit is shown in Fig. 2. It is built around a 5V voltage regulator 7805 (IC1), microcontroller ATmega16 (IC2), 6V voltage regulator 7806 (IC3), motor driver L293D (IC4), four 1N4007 diodes (D1 through D4), 10-segment bar LED array (BAR1), three LEDs (LED1 through LED3), two 6V battery-operated motors, and a few other components.

The sensor circuit unit is shown in Fig. 3. It is built around two LM324 op-amps (IC5 and IC6), eight IR transmitting LEDs (IRTX1-IRTX8), eight IR receiving LEDs (IRRX1-IRRX8), and a few other components.

The line data is read by eight pairs of IR transmitting LEDs and IR receiving LEDs (used as sensors) arranged in a line. The resistance of the IR sensor varies with the reflection of IR light from the colour of tracks on the floor,

with white colour track reflecting the most and black colour track reflecting the least.

The change in the resistance is obtained when the voltage is changed using a resistance network and fed to eight op-amps (four each from IC5 and IC6) to further amplify the signal.

The outputs of the op-amps are input signals for the microcontroller ATmega16. Thus, the status of signals obtained by the microcontroller is displayed on a 10-LED bar array (used only eight out of ten LEDs here).

The signal can be inverted by using jumper J1, which enables the robot to follow a white line on a black background, or black line on a white background, on the floor.

Construction and testing

An actual-size PCB layout for the main circuit is shown in Fig. 4 and its components layout in Fig. 5. Assemble the circuit on the PCB. Connect a LiPo battery across BATT.1. You can also use a normal 12V lead-acid battery in place of LiPo battery. Before mounting the ATmega16A on the PCB, do not forget to upload `custom_pid_1f.hex` into it.

An actual-size PCB layout for the sensor circuit is shown in Fig. 6 and its components layout in Fig. 7. After assembling the circuit on the PCB, connect CON3 to CON4.

After assembling the main and sensor units on the respective PCBs, fix the main unit on top of the chassis along with the battery and sensor unit at the bottom of the chassis (around 25mm above the surface).

For more details on the assembly and connections, please see author's prototype in Fig. 1 and the sensor unit assembly in Fig. 8.

Initial setup

Fix two battery-operated motors, a castor wheel, and a battery on a good chassis, as explained below. Place the eight-IR-sensor array board on the front side with a gap of about 5mm between the floor and IR LEDs. Next, place the main circuit board on the chassis and write/burn the `custom_pid_1f.hex` file to the microcontroller (ATmega16) through ISP port using any suitable AVR programmer after switching on the power supply to the board from the battery. Now, switch